CHAPITRE 3

Université Alioune Diop de Bambey
L'excellence est ma constance, l'éthique ma vertu

ARCHITECTURE & TECHNOLOGIE DES ORDINATEURS

Le codage binaire

1

Dr Abdou Khadre DIOP

Département Technologies de l'Information et de la Communication

INTRODUCTION (1/1)

Le codage binaire consiste à utiliser deux états (représentés par les chiffres 0 et 1) pour coder les informations. C'est donc avec ce codage que fonctionnent les ordinateurs.

Ainsi pour un code de n bits, il est possible de représenter 2^n valeurs.

- Exemple : Pour 3 bits il est possible de représenter 8 valeurs.

CHAPITRE 3

Bits	A	B	C	D
Poids	8	4	2	1
0	0	0	0	0
1	0	0	0	1
2	0	0	1	0
3	0	0	1	1
4	0	1	0	0
5	0	1	0	1
6	0	1	1	0
7	0	1	1	1
8	1	0	0	0
9	1	0	0	1
10	1	0	1	0
11	1	0	1	1
12	1	1	0	0
13	1	1	0	1
14	1	1	1	0
15	1	1	1	1

LE CODAGE BINAIRE NATUREL (1/1)

Ce code utilise l'expression naturelle du nombre en base 2 (système de numération binaire). Il est donc intéressant pour effectuer opérations arithmétiques. On dit que ce code est un code pondéré (le $i^{ème}$ bit en partant de la droite "pèse" 2^{i-1})

- Exemple : Code binaire naturel à 4 bits

CHAPITRE 3

Décimal	BCD			
0	0	0	0	0
1	0	0	0	1
2	0	0	1	0
3	0	0	1	1
4	0	1	0	0
5	0	1	0	1
6	0	1	1	0
7	0	1	1	1
8	1	0	0	0
9	1	0	0	1

LE CODE BCD (BINARY CODED DECIMAL (1/2)

Le code BCD est l'un des codes les plus répandus pour représenter les chiffres décimaux en binaire. Chaque chiffre décimal est codé individuellement en son équivalent binaire sur 4 bits.

- Exemple : Coder en BCD $(307)_{10}$

$$(307)_{10} = (0011\ 0000\ 0111)_{\text{BCD}}$$

Quatre bits sont nécessaires pour coder les dix chiffres décimaux. Mais les valeurs représentables sur 4 bits sont au nombre de $2^4 = 16$. Il y a donc 6 configurations inutilisés. Il faut en tenir compte pour les opérations arithmétiques.

Pour l'addition, il faut ajouter 6 chaque fois que le résultat est > 9.

Décimal	BCD			
0	0	0	0	0
1	0	0	0	1
2	0	0	1	0
3	0	0	1	1
4	0	1	0	0
5	0	1	0	1
6	0	1	1	0
7	0	1	1	1
8	1	0	0	0
9	1	0	0	1

LE CODE BCD (BINARY CODED DECIMAL (2/2)

Exemple : Calculer $(19)_{10} + (12)_{10} = (31)_{10}$

$(19)_{10} = (10011)_2$ et $(12)_{10} = (1100)_2$

$$+\ \begin{array}{r}(10011)_2\\ (1100)_2\end{array}$$

$$(11111)_2 = (31)_{10}$$

$$+\ \begin{array}{r}(0001\ 1001)_{\text{BCD}}\\ (0001\ 0010)_{\text{BCD}}\end{array}$$

$$\begin{array}{r}(0010\ 1011)_{\text{BCD}}\\ (\quad\ 0110)_{\text{BCD}}\end{array}$$

$$(31)_{10} = (0011\ 0001)_{\text{BCD}}$$

Le code BCD est un code pondéré dont les poids sont 8, 4, 2 et 1.

LE CODE EXCES-3 (1/1)

Décimal	Excès-3			
0	0	0	1	1
1	0	1	0	0
2	0	1	0	1
3	0	1	1	0
4	0	1	1	1
5	1	0	0	0
6	1	0	0	1
7	1	0	1	0
8	1	0	1	1
9	1	1	0	0

Le code Excès-3 n'est pas un code pondéré : chaque chiffre decimal est code en son equivalent binaire +3. On dit aussi que c'est un code autocomplémentaire.

Exemple : Coder $(307)_{10}$ en Excès $-$ 3

$$(307)_{10} = (0110\ 0011\ 1010)_{Excès-3}$$

CHAPITRE 3

Décimal	Aiken			
0	0	0	0	0
1	0	0	0	1
2	0	0	1	0
3	0	0	1	1
4	0	1	0	0
5	1	0	1	1
6	1	1	0	0
7	1	1	0	1
8	1	1	1	0
9	1	1	1	1

LE CODE AIKEN (1/1)

Le code Aiken est un code autocomplémentaire dont les poids binaires sont 2, 4, 2 et 1.

Exemple : Coder $(307)_{10}$ en Aiken

$$(307)_{10} = (0011\ 0000\ 1101)_{Aiken}$$

CHAPITRE 3

LE CODE ASCII (1/1)

LSB \ MSB		0 000	1 001	2 010	3 011	4 100	5 101	6 110	7 111
0	0000	NUL	DLE	SP	0	@	P	`	p
1	0001	SOH	DC1	!	1	A	Q	a	q
2	0010	STX	DC2	"	2	B	R	b	r
3	0011	ETX	DC3	#	3	C	S	c	s
4	0100	EOT	DC4	$	4	D	T	d	t
5	0101	ENQ	NAK	%	5	E	U	e	u
6	0110	ACK	SYN	&	6	F	V	f	v
7	0111	BEL	ETB	'	7	G	W	g	w
8	1000	BS	CAN	(	8	H	X	h	x
9	1001	HT	EM	)	9	I	Y	i	y
A	1010	LF	SUB	*	:	J	Z	j	z
B	1011	VT	ESC	+	;	K	[	k	}
C	1100	FF	FS	,	<	L	\	l	|
D	1101	CR	GS	-	=	M	]	m	{
E	1110	SO	RS	.	>	N	^	n	~
F	1111	SI	US	/	?	O	_	o	DEL

L'*American Standard Code for Information Interchange*, de son petit nom le code ASCII ou Ascii (prononcez aski), est une norme informatique pour le codage des caractères. En adoptant le même codage, les systèmes informatiques conçus par n'importe quel fabricant savent ainsi échanger du texte, des nombres, des signes de ponctuation et bien d'autres symboles.

C H A P I T R E 3

Introduction :

Les informations sont stockées sous forme binaire dans un ordinateur. Elles peuvent subir des modifications involontaires lors de leur transmission ou lors de leur stockage en mémoire.

Il faut donc utiliser des codes permettant de détecter ou même de corriger les erreurs dues à ces modifications. Ces codes portent sur un nombre de bits supérieur à celui strictement nécessaire pour coder l'information.

Aux m bits de données, on ajoute k bits de contrôle, soit n =m+k bits qui vont être transmis ou stockés en mémoire.

Certains codes ne permettent que la détection des erreurs (Codes autovérificateurs), d'autres permettent la détection et la correction d'une ou plusieurs erreurs (Codes autocorrecteurs).

C H A P I T R E 3

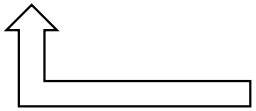
Le contrôle de parité :

Le contrôle de parité est le code autovérificateur le plus simple. Aux m bits de données, on ajoute 1 bit de contrôle appelé bit de parité.

Le bit de parité représente le bit le plus significatif. Sa valeur est telle que le nombre total de bits à 1 calculé sur (m+1) bits est pair (dans le cas d'une parité paire) ou impair (parité impaire).

Exemple : Transmission entre un ordinateur et un terminal de caractères codés en ASCII (7 bits)+1 bit de parité.

A	➔	11000001
B	➔	11000010
C	➔	01000011
D	➔	11000100
E	➔	01000101



Bit de parité impaire

CHAPITRE 3

Le contrôle de parité :

Si un bit est erroné pendant le transfert, la parité n'est plus vérifiée. L'erreur est détectée : il faut alors retransmettre l'information.

S'il y a deux bits erronés pendant le transfert d'un même caractère, la parité est vérifiée mais la détection est impossible.

Le contrôle de parité ne peut etre utilisé que pour des transmissions où le taux d'erreurs est très faible (Exemple : A l'intérieur d'un ordinateur ou entre un ordinateur et ses périphériques).